Name: Syed Zaeem Ali Shah

Student ID: 100874830

Module Leader: Professor Dr. Suleiman Aliyu

Module Title: Distributed Applications and Web Services

Course Code: (6CC545)

Coursework 1 Commentary

Bachelor of Science in Computing & Information Technology

University of Derby, School of Computing

Submission Date: 29 October, 2026

Word Count: 1160 words

Originality Statement

I, Syed Zaeem Ali Shah, Student ID: 100874830 declare that this assessment is my own work and that I have correctly acknowledged the work of others using the Harvard referencing. This assessment is in accordance with University guidance on good academic conduct.

Distributed Service Foundation and Remote Invocation Prototype

Problem, Scope and Requirements

The proposed two-node Go prototype project tests communication between two independent processes before developing a large decentralised file sharing system. Each node has its own unique ID, a configurable address and its own local state. The two nodes have used REST, TCP, UDP and gRPC to communicate data in different ways. JSON and Protocol Buffers explain how the data has been shared through these two interfaces. This Go prototype is limited to two nodes. The design makes it easy to understand and test each communication method separately within the given scope. Starting with two nodes provides a clear way to study communication, remote invocation and data exchange before introducing more complex distributed-system features (Coulouris et al., 2012). However, this prototype does not prove how the system will operate with many peers. For example, one node can request file metadata from another, but it does not show how the system will manage conflicting data with additional peers. This limitation highlights the need for further development to support additional peers. Moreover, Docker, TLS, JWT authentication, gossip and secure file transfer are outside the current scope. I consider this design is a strong foundation for the next stage, where additional peers, security controls and peer-to-peer features will be developed and tested in Coursework Two.

Architecture and Platform Choice

The architecture and platform for this prototype support the required communication methods. Go supports HTTP, TCP, UDP, JSON and concurrent operations through its standard library. It helps me to implement and test the required functions without depending on a large framework. Coulouris et al. (2012) highlights the importance of communication and process coordination in distributed systems which makes these features relevant to this prototype. It has used a command-line entry point to start each node and load its configuration. Also, separate components manage network communication, service operations and local state. This design allows individual components to be tested independently. However, separate component boundaries alone do not ensure that the design is fully reliable. The node needs to manage its complete lifecycle, including startup, listener failures, request handling, cancellation and

shutdown. For example, if one listener fails to start after another has started, the application must report the error and stop the running service. It shows why startup, error handling and shutdown are important elements to be considered as part of the architecture.

The current design does not address all the challenges of managing information because it has only a limited communication scope. It avoids unnecessary dependence on shared state store, but it can also lead to differences between nodes. Go provides mutexes and context cancellation, but these features must be used correctly. For example, concurrent requests may cause a data race if they access and update a shared map without proper protection. I have used the Go race detector and concurrent tests to identify such problems. Overall, Go suits this prototype, but its main limitation is that developers must manage concurrency and service lifecycles carefully.

REST and State Design

I have used REST to expose node health and selected state through HTTP. GET retrieves information, whereas POST performs a defined state-changing operation. JSON makes requests and responses easy to inspect with tools such as curl. It is useful during development because I can examine the information sent between the client and server. The HTTP handler should validate a request before changing state. The application should also check the content type and limit request-body size. These checks prevent invalid or unnecessarily large requests from reaching the state layer. The concurrency safe registry protects shared metadata with a mutex. Returning copied snapshots also prevents callers from changing internal data accidentally. Deterministic ordering makes responses easy to compare and tests. A useful negative test sends an invalid POST and checks that the stored state remains unchanged. Returning an error is not enough if the operation has already damaged the data. This test checks both request validation and state protection. REST is suitable for simple HTTP access, but it does not provide authentication or consistency between nodes. These require additional design and testing.

TCP, UDP and gRPC

TCP and UDP serve different needs. TCP provides reliable, ordered delivery, but it treats data as a byte stream rather than separate application messages (Eddy, 2022). The prototype needs explicit message framing, input-length validation and deadlines. Without framing, the receiver

may not find out where a message ends. A successful round trip shows that the nodes can exchange and decode a message. An oversized-message test checks whether invalid input is rejected without crashing the node.

In contrast, UDP sends separate datagrams and has less protocol overhead, but delivery and ordering are not guaranteed (Postel, 1980). It is useful for demonstrating lightweight communication between two processes. However, it would not be suitable for complete file transfer without additional reliability, sequencing and integrity checks. Failure handling is also different. A successful TCP transfer does not prove that the requested operation has finished. Similarly, a missing UDP response does not prove that the receiver failed to process the request. These features need careful design because neither transport alone can make sure that the application operation happens exactly once.

I have also used gRPC with a Protocol Buffer service contract. The client requests file metadata from the other node, and the response demonstrates remote invocation. The .proto definition specifies the method and message fields. Generated client and server bindings help both sides follow the same contract. gRPC provides a defined contract and generated code, which can reduce differences between clients and servers. However, gRPC requires code generation and extra setup. It can also fail when the remote service is unavailable. A negative test confirms that the client receives a clear error within its deadline.

Testing and Failure Evidence

Positive tests show whether the communication paths work. A successful REST request confirms that the endpoint returns the expected information. TCP and UDP round trips prove that the nodes can exchange messages. A gRPC response confirms that the client can obtain metadata from the remote node. Negative tests provide different evidence. Malformed and oversized messages are rejected. The unavailable gRPC service returns a clear error. Shutdown tests also confirm that listeners stop correctly. These results support the design, but successful local tests cannot prove production level reliability. Automated assertions, repeated runs and Go’s race detector can reveal problems that a single successful test may miss.

Critical Evaluation and Coursework Two Handoff

The prototype brings several communication methods into one working application. Its separate components, local state and defined service contracts provide a useful base for further development. However, it is not yet a secure peer-to-peer file-sharing service. Two nodes cannot prove the full challenges of discovery, conflicting metadata and recovery from large network failures. Coursework Two will introduce event publishing and subscription, containers, TLS, authentication and authorisation. Event handling will need rules for duplicate and missed messages. Gossip-based discovery will also need to manage outdated peer information. I would introduce these features in Coursework 2, and retest the existing interfaces after each change.

References

Coulouris, G., Dollimore, J., Kindberg, T. and Blair, G. (2012) Distributed Systems: Concepts and Design. 5th edn. Harlow: Pearson.

Eddy, W. (ed.) (2022) Transmission Control Protocol (TCP). RFC 9293. Internet Engineering Task Force. Available at: https://www.rfc-editor.org/rfc/rfc9293.html

Postel, J. (1980) User Datagram Protocol. RFC 768. Internet Engineering Task Force. Available at: https://www.rfc-editor.org/rfc/rfc768.html